PROGRAMAÇÃO FRONT-END

SEMANA 18

CI/CD, testes automatizados e testes de performance

3 aulas de 50 minutos

Parte 1 - Roteiro teórico do professor
Parte 2 - Atividade prática dos estudantes
Parte 3 - Orientações, gabarito e plano B

Proposta didática
Material adaptado para estudantes do Ensino Médio Técnico. A primeira aula concentra os conceitos essenciais de CI/CD, testes automatizados e performance. As duas aulas seguintes trabalham uma prática progressiva, sem exigir instalação local de Jest, Cypress, JMeter ou k6: os estudantes usam JavaScript simples no navegador, DevTools e Lighthouse, mantendo o princípio pedagógico dos materiais originais.

PARTE 1 - ROTEIRO TEÓRICO DO PROFESSOR

Aula 1 - 50 minutos | CI/CD, testes automatizados e performance no front-end

1. Objetivos de aprendizagem

- Compreender por que projetos front-end utilizam CI/CD para automatizar verificações e entregas.
- Diferenciar integração contínua (CI) e entrega/deploy contínuo (CD) em nível introdutório.
- Reconhecer as etapas básicas de um pipeline: obter código, testar, gerar build e publicar.
- Compreender a função de testes unitários, de integração e ponta a ponta.
- Reconhecer Jest e Cypress como ferramentas de testes automatizados apresentadas no material.
- Entender o objetivo dos testes de performance e o papel de Lighthouse, JMeter ou k6.
- Relacionar qualidade, automação, testes e experiência do usuário.

2. Conhecimentos prévios

Espera-se que os estudantes reconheçam HTML, CSS e JavaScript básicos, saibam abrir o Console/DevTools do navegador e tenham contato inicial com Git/GitHub. Não é necessário dominar automação, testes ou pipelines antes desta sequência.

3. Recursos

- Projetor e lousa.
- Laboratório de informática com navegador Google Chrome ou compatível.
- Editor de código.
- Internet para uso do Lighthouse e, quando possível, acesso ao GitHub.
- Um projeto front-end já publicado ou uma URL de exemplo fornecida pelo professor.

4. Cronograma das três aulas

Aula	Tipo	Foco
1 - 50 min	Teórica	CI/CD, pipelines, testes automatizados, Jest/Cypress e performance.
2 - 50 min	Prática	Testes simples de lógica no navegador + leitura de um pipeline simplificado.
3 - 50 min	Prática	Auditoria com Lighthouse, análise de gargalos, melhoria e comparação de resultados.

5. Ponto de partida: por que automatizar?

Apresente o cenário do material: uma equipe front-end atualiza um e-commerce com frequência, mas testar, compilar e publicar manualmente consome tempo e aumenta a chance de falhas chegarem ao usuário.

Perguntas para iniciar:

- O que pode acontecer quando uma equipe publica alterações sem testar?
- Quais tarefas repetitivas poderiam ser automatizadas?
- É melhor descobrir um erro antes ou depois de o usuário acessar o site?

6. O que é CI/CD?

CI/CD reúne práticas de automação utilizadas para integrar mudanças, verificar qualidade e preparar ou realizar entregas com mais frequência e previsibilidade.

Analogia - linha de produção com pontos de controle
Imagine uma linha de produção. Antes de o produto chegar ao cliente, ele passa por verificações. No CI/CD, cada alteração de código pode passar por uma sequência automática de conferências: buscar o código, executar testes, gerar a versão pronta e, quando configurado, publicar.

```
ALTERAÇÃO NO CÓDIGO
        ↓
INTEGRAÇÃO (CI)
        ↓
TESTES
        ↓
BUILD
        ↓
ENTREGA / DEPLOY (CD)
```

7. CI - Integração Contínua

Integração Contínua significa integrar alterações ao repositório com frequência e verificar automaticamente se essas mudanças quebraram algo importante.

Exemplo simples: um estudante altera `script.js`, faz um push e um pipeline verifica se os arquivos principais existem e se o JavaScript não possui erro de sintaxe.

8. CD - Entrega e deploy

CD está relacionado à preparação e/ou publicação automatizada de versões após as verificações. Para esta sequência, trate o conceito de forma introdutória: depois que o código passa pelos testes e pelo build, ele pode ser preparado ou enviado para um ambiente de publicação.

Analogia
CI é como conferir se todas as peças estão corretas durante a montagem. CD é como preparar e encaminhar o produto aprovado para chegar ao usuário.

9. Etapas de um pipeline

- Checkout: obter o código do repositório.
- Instalação/configuração: preparar o ambiente.
- Testes: verificar se o comportamento esperado continua correto.
- Build: gerar a versão preparada para publicação.
- Deploy: publicar a versão aprovada, quando essa etapa estiver configurada.

10. Exemplo didático de pipeline

O exemplo abaixo é propositalmente simples. Ele verifica um projeto estático e a sintaxe do JavaScript no ambiente do GitHub Actions. Não é necessário executá-lo na aula prática se a turma ainda não estiver pronta para configurar workflows.

```yaml
name: Verificacao Front-end

on:
  push:
    branches: [main]

jobs:
  verificar:
    runs-on: ubuntu-latest
    steps:
```

```
      - uses: actions/checkout@v4
      - name: Verificar sintaxe do JavaScript
        run: node --check script.js
```

11. Testes automatizados: qual é a ideia?

Teste automatizado é um código ou procedimento configurado para verificar automaticamente se uma parte da aplicação apresenta o resultado esperado. O objetivo é descobrir regressões mais cedo e reduzir a dependência de conferências manuais repetitivas.

Analogia - conferir uma lâmpada antes de instalar
Testar uma função isolada é como verificar uma lâmpada antes de colocá-la no circuito. Depois, também precisamos verificar se lâmpada, interruptor e fiação funcionam juntos. Essa diferença ajuda a introduzir testes unitários e de integração.

12. Teste unitário

Teste unitário verifica uma parte pequena e isolada da lógica, como uma função de cálculo. O material apresenta Jest como ferramenta adequada para esse tipo de teste.

```
function calcularTotal(valores) {
    return valores.reduce((total, valor) => total + valor, 0);
}

// Exemplo didático de teste no navegador
console.assert(calcularTotal([100, 50]) === 150, "Teste falhou");
```

Explique que `console.assert` será usado apenas como ponte didática na prática por não exigir instalação. Em projetos profissionais, frameworks como Jest oferecem organização e recursos próprios para testes.

Exemplo equivalente em Jest, apenas para compreensão:

```
test("calcula o total do carrinho", () => {
    expect(calcularTotal([100, 50])).toBe(150);
});
```

13. Testes de integração e ponta a ponta

Testes de integração verificam se partes diferentes funcionam corretamente em conjunto. Testes ponta a ponta (E2E) simulam uma jornada mais completa do usuário. O material apresenta Cypress como uma ferramenta apropriada para validar interações da interface e fluxos do usuário.

Analogia - sistema completo
Depois de testar a lâmpada isoladamente, é preciso testar o conjunto: interruptor, fiação e lâmpada. Em um e-commerce, podemos testar carrinho + cálculo do total + checkout trabalhando em conjunto.

14. Jest x Cypress

Ferramenta	Uso introdutório nesta aula
Jest	Testes unitários: verificar funções ou componentes isolados.
Cypress	Testes de integração/E2E: verificar interações e jornadas na interface.

15. Por que testes evitam regressões?

Regressão ocorre quando uma alteração nova faz uma funcionalidade que já funcionava apresentar problema. Testes automatizados ajudam a verificar novamente comportamentos importantes sempre que o código muda.

Exemplo do material: uma atualização visual pode, sem intenção, afetar datas, cálculos ou outras funcionalidades estáveis.

16. Testes de performance

Testes de performance avaliam a eficiência da aplicação, por exemplo tempo de carregamento, capacidade de resposta e comportamento sob diferentes níveis de uso.

Analogia - fila de supermercado
Uma loja pode funcionar bem com 10 clientes e ficar lenta com 500. Testes de carga procuram descobrir como o sistema reage quando a quantidade de acessos aumenta e onde aparece o gargalo.

17. Lighthouse

Lighthouse é uma ferramenta disponível no ecossistema do Chrome que gera uma auditoria da página. No material da semana, ele é utilizado para observar performance e identificar oportunidades de melhoria.

O material cita métricas como LCP, TTI e FID. Na prática, oriente os estudantes a registrar as métricas realmente exibidas pela versão do Lighthouse disponível no laboratório e as recomendações apresentadas pela ferramenta.

18. JMeter e k6

O material apresenta Apache JMeter ou k6 para simular níveis de carga, observando tempo de resposta e taxa de erros. Como essas ferramentas podem exigir instalação e preparação adicional, mantenha sua função conceitual nesta sequência e use uma análise simulada na atividade prática.

19. Gargalo de performance

Gargalo é um ponto que limita o desempenho do sistema. Em front-end, exemplos podem incluir imagens muito pesadas, JavaScript excessivo, recursos desnecessários ou respostas lentas de serviços externos.

Analogia
Se uma loja tem cinco corredores, mas apenas um caixa aberto, o caixa pode virar o gargalo. No site, um único recurso muito pesado também pode atrasar toda a experiência.

20. O que colocar na lousa

```
CI/CD
Código → Testes → Build → Deploy

CI = integrar e verificar mudanças
CD = preparar/realizar entregas

TESTE UNITÁRIO = parte isolada
TESTE DE INTEGRAÇÃO = partes trabalhando juntas
E2E = jornada do usuário

Jest → testes unitários
Cypress → integração/E2E

PERFORMANCE
Lighthouse → auditoria da página
JMeter/k6 → carga
```

```
GARGALO = ponto que limita o desempenho
```

21. Perguntas rápidas

- Qual problema o CI/CD ajuda a reduzir?
- Qual é a diferença básica entre CI e CD?
- O que um teste unitário verifica?
- Quando um teste de integração é útil?
- Por que um e-commerce deve testar performance?
- O que significa encontrar um gargalo?

22. Respostas esperadas

- Automatizar verificações e entregas reduz tarefas manuais repetitivas e ajuda a detectar falhas antes da publicação.
- CI integra/verifica mudanças; CD está relacionado à preparação e/ou entrega/publicação de versões aprovadas.
- Um teste unitário verifica uma parte pequena e isolada da lógica.
- Integração verifica se partes diferentes funcionam juntas.
- Performance afeta tempo de resposta e experiência do usuário, principalmente com maior uso.
- Gargalo é um ponto que limita o desempenho do restante do sistema.

PARTE 2 - ROTEIRO DA ATIVIDADE PRÁTICA

Aulas 2 e 3 - 100 minutos | Qualidade e performance de uma aplicação front-end

1. Identificação

Nome	Turma	Data
Yasmin silva	3b	24/08/26

2. Situação-problema

A empresa fictícia WebFast precisa reduzir falhas e lentidão antes de publicar novas versões de seu e-commerce. Sua dupla deverá verificar uma pequena lógica de carrinho, observar a qualidade do código e executar uma auditoria de performance em uma página front-end já publicada ou em uma URL fornecida pelo professor.

3. Objetivos da atividade

- Compreender o princípio de um teste automatizado simples.
- Executar verificações de lógica sem depender de instalação de bibliotecas.
- Utilizar o Lighthouse para analisar uma página front-end.
- Identificar possíveis gargalos e propor melhorias.
- Relacionar testes, performance e CI/CD.

4. Materiais

- Computador com navegador Chrome ou compatível.
- Editor de código.
- Console/DevTools.
- Página front-end publicada ou URL disponibilizada pelo professor.
- Internet para a auditoria com Lighthouse.

AULA 2 - PARTE 1 - 50 MINUTOS

5. Etapa 1 - Entendendo a função a ser testada

Crie dois arquivos na mesma pasta: `index.html` e `script.js`. O `index.html` será responsável por carregar o JavaScript no navegador.

Arquivo index.html:

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Testes Front-End</title>
</head>
<body>
    <h1>Testes JavaScript</h1>
    <script src="script.js"></script>
</body>
</html>
```

Depois, abra o `index.html` no navegador e pressione F12 para acompanhar os resultados no Console.

```
function calcularTotal(valores) {
    return valores.reduce((total, valor) => total + valor, 0);
}
```

```
console.log(calcularTotal([100, 50]));
```

Resultado esperado: 150.

6. Etapa 2 - Criando testes simples no navegador

Acrescente verificações automáticas usando `console.assert`:

```
console.assert(
    calcularTotal([100, 50]) === 150,
    "Teste 1 falhou"
);

console.assert(
    calcularTotal([20, 30, 10]) === 60,
    "Teste 2 falhou"
);

console.log("Testes concluídos");
```

Abra o Console do navegador. Se nenhuma mensagem de falha aparecer, os resultados esperados foram atendidos.

Importante
Esta é uma adaptação didática para demonstrar o princípio de teste automatizado sem instalar Jest. O material da semana apresenta Jest como ferramenta profissional para testes unitários e Cypress para testes de integração/E2E.

7. Etapa 3 - Faça um teste falhar de propósito

Altere temporariamente o resultado esperado do primeiro teste para 999 e observe o Console. Depois, corrija o valor novamente.

O que aconteceu quando o valor esperado estava errado?

_____

Quando o valor esperado foi alterado para 999, o teste apresentou uma mensagem de falha no Console, porque o resultado real da função era 150. Depois que o valor esperado voltou para 150, o teste passou novamente._____

8. Etapa 4 - Relacione com CI/CD

Complete o fluxo com as palavras: TESTES, BUILD, DEPLOY.

```
CÓDIGO
   ↓
TESTES
   ↓
BUILD
   ↓
DEPLOY
```

9. Etapa 5 - Leitura de um pipeline simplificado

Observe o exemplo abaixo. Ele seria executado no ambiente do GitHub Actions, não no computador local:

```
name: Verificacao Front-end

on:
  push:
    branches: [main]
```

```
jobs:
  verificar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Verificar sintaxe do JavaScript
        run: node --check script.js
```

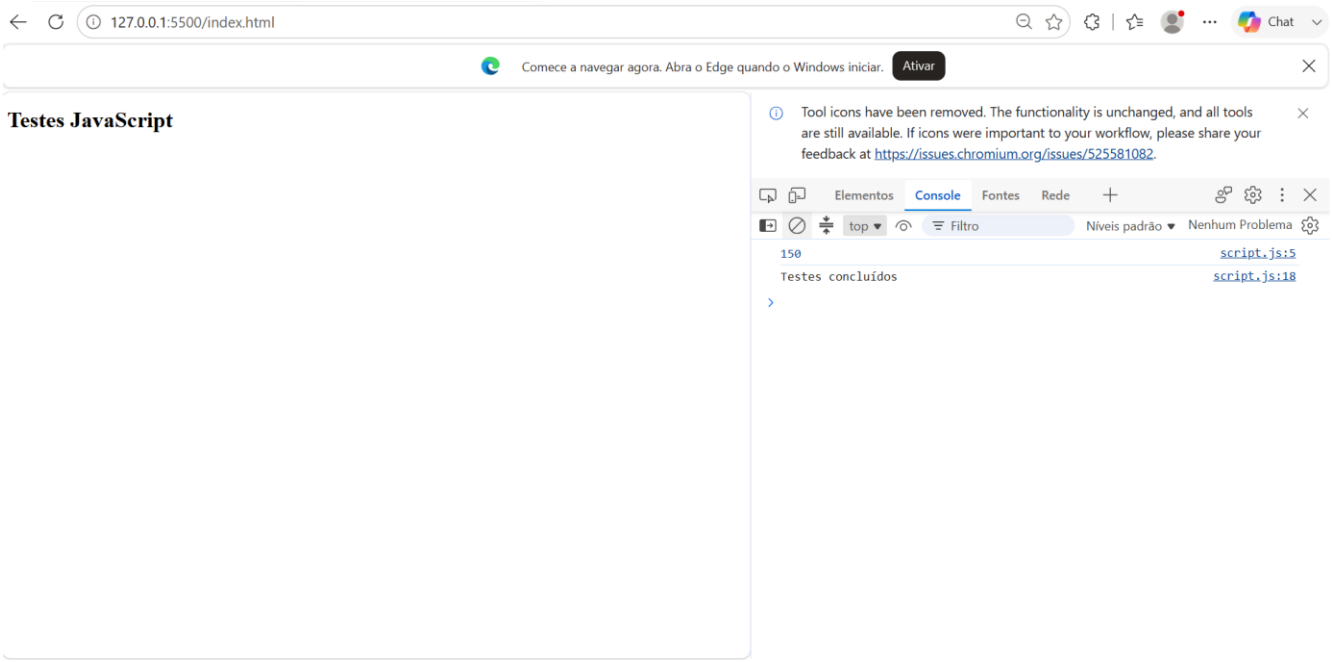
Responda: qual parte desse exemplo realiza uma verificação automática?

A parte que realiza a verificação automática é node --check script.js, porque ela verifica automaticamente a sintaxe do arquivo JavaScript

10. Evidência 1 - Testes no Console

Insira um print do Console mostrando `Testes concluídos` e sem mensagens de falha.

[ESPAÇO PARA PRINT]



AULA 3 - PARTE 2 - 50 MINUTOS

11. Etapa 6 - Escolha da página para auditoria

Utilize uma página front-end já publicada por sua dupla, um projeto de aula anterior ou uma URL fornecida pelo professor. Se o laboratório não permitir publicação, o professor deverá disponibilizar uma página de demonstração.

12. Etapa 7 - Executando o Lighthouse

- Abra a página no Google Chrome.
- Abra o DevTools.
- Localize a ferramenta Lighthouse.
- Execute uma auditoria com foco em Performance.
- Aguarde o relatório terminar.
- Registre a pontuação e as métricas/recomendações exibidas.

13. Registro da primeira auditoria

Item	Resultado observado
Pontuação de Performance	99

LCP, se exibido	**0,6 s**
Outra métrica relevante exibida	FCP: 0,6 s
Principal recomendação do Lighthouse	Até agora, nenhuma recomendação crítica aparece, pois o desempenho está muito bom.
Segundo ponto de melhoria	O **Total Blocking Time (TBT)** é de **90 ms**, então uma possível melhoria seria reduzir ainda mais tarefas JavaScript que possam bloquear a página.

14. Etapa 8 - Identificando gargalos

Analise as recomendações do relatório e marque o que apareceu ou poderia afetar a página:

☐ Imagens grandes/pesadas.

☐ **JAVASCRIPT OU CSS EM EXCESSO.(ESSA)**

☐ Recursos que demoram para carregar.

☐ Muitos arquivos/requisições.

☐ Outro ponto indicado pela ferramenta.

Qual parece ser o principal gargalo ou oportunidade de melhoria?

A principal oportunidade de melhoria é reduzir possíveis bloqueios causados pelo JavaScript, pois o Total Blocking Time (TBT) foi de 90 ms. Apesar disso, o site apresentou excelente desempenho, com pontuação de 99 no Lighthouse.

15. Etapa 9 - Uma melhoria possível

Escolha uma melhoria simples. Se vocês tiverem acesso ao código, façam a alteração. Se não tiverem, descrevam exatamente o que fariam.

☐ Otimizar ou substituir uma imagem muito pesada.

☐ Remover recurso que não é necessário.

☐ **REDUZIR CÓDIGO NÃO UTILIZADO.**

☐ Evitar carregamento de conteúdo desnecessário.

☐ Outra recomendação indicada pelo Lighthouse.

Melhoria escolhida:

A melhoria escolhida foi revisar e reduzir códigos JavaScript que não são necessários ou que podem causar bloqueios durante o carregamento da página. Com essa otimização, espera-se reduzir ainda mais o Total Blocking Time e manter ou melhorar o desempenho do site.

16. Etapa 10 - Segunda auditoria

Se a página puder ser alterada, execute o Lighthouse novamente e compare. Caso não seja possível editar, registre a melhoria proposta e explique qual resultado espera obter.

Comparação	Antes	Depois / esperado
Performance	99	100
Principal métrica observada	0,6 s	0,8 s

Gargalo/recomendação	Até agora, nenhuma recomendação crítica aparece, pois o desempenho está muito bom	Sem nenhuma.

17. Etapa 11 - Simulação de teste de carga

O material apresenta JMeter ou k6 para simular muitos usuários. Nesta atividade, interprete o cenário simulado abaixo:

Usuários simultâneos	Tempo médio de resposta	Taxa de erros
50	0,8 s	0%
200	1,6 s	1%
500	4,9 s	8%

A partir de qual nível a aplicação começa a apresentar sinais claros de degradação?

A partir de 500 usuários simultâneos, pois o tempo de resposta aumenta para 4,9 segundos e a taxa de erros chega a 8%.

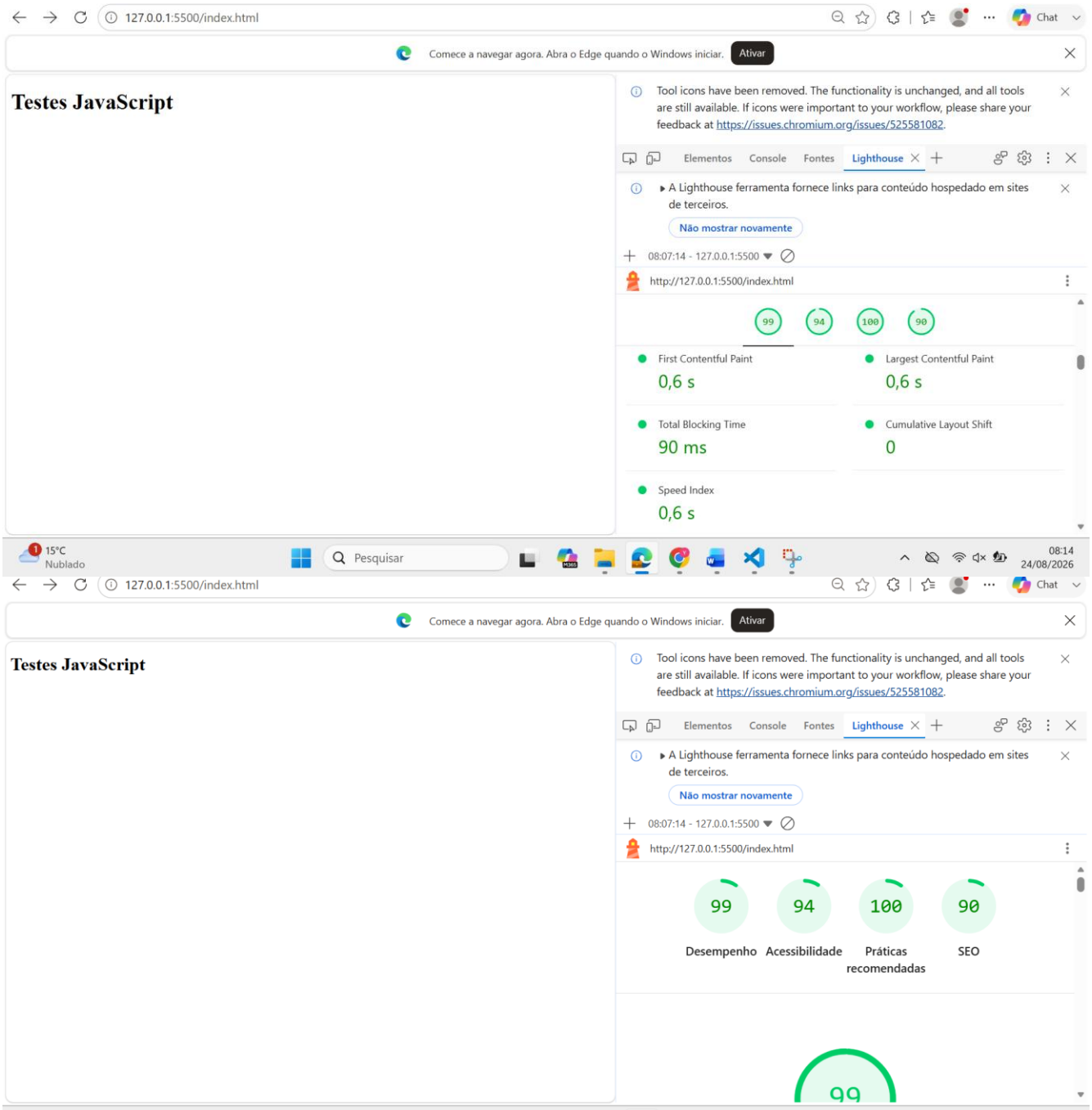
Que conclusão a equipe pode tirar desse teste?

A aplicação apresenta bom desempenho com até 200 usuários simultâneos, mas começa a ter problemas com 500 usuários. Portanto, a equipe deve otimizar a aplicação e aumentar sua capacidade para suportar uma quantidade maior de usuários.

18. Evidência 2 - Lighthouse

Insira um print do relatório do Lighthouse com a pontuação de Performance visível.

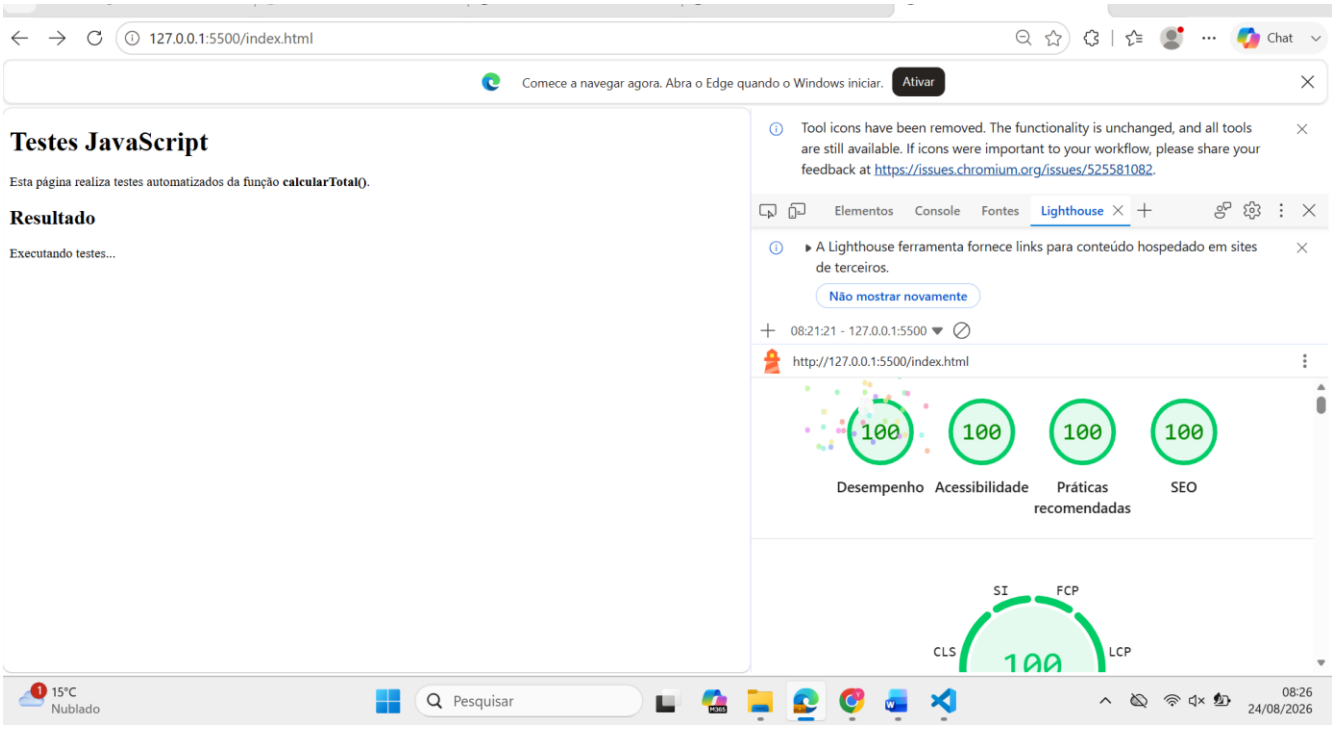
[ESPAÇO PARA PRINT]

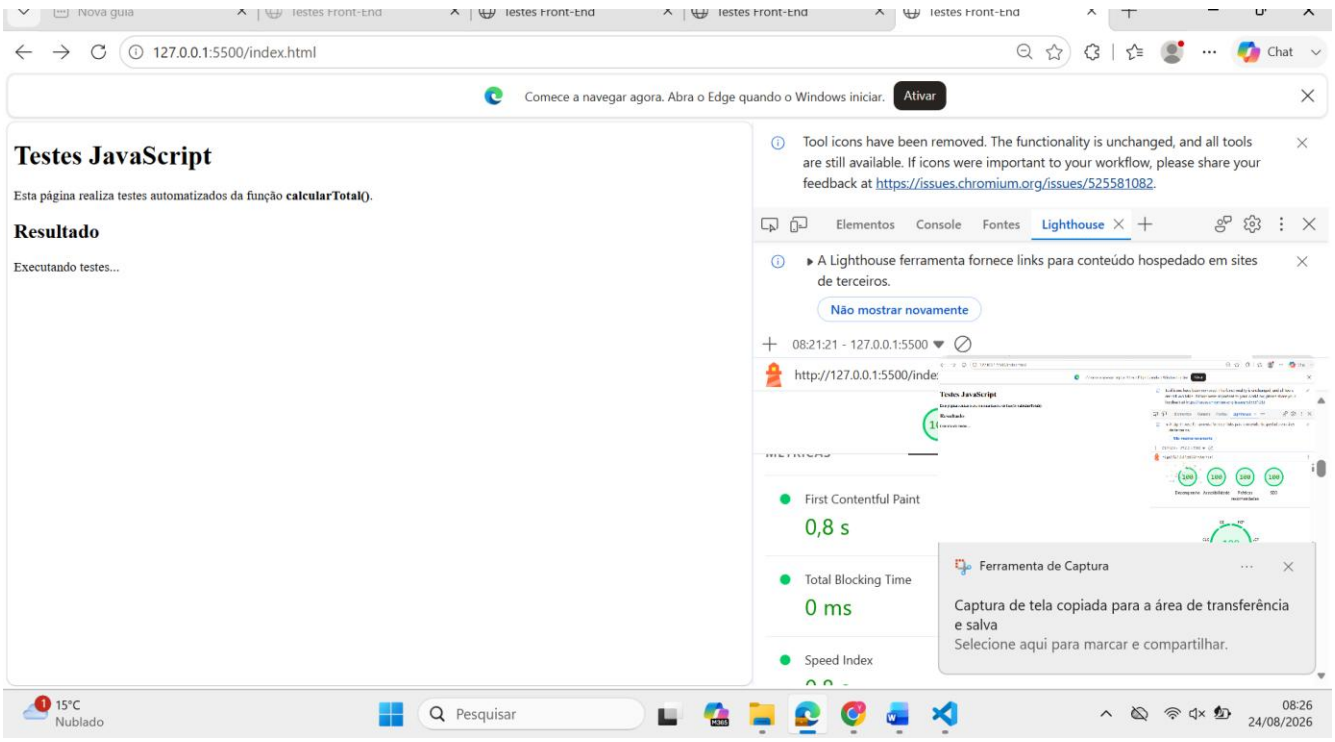


19. Evidência 3 - Gargalo ou melhoria

Insira um print da recomendação analisada ou da comparação antes/depois.

[ESPAÇO PARA PRINT]





20. Perguntas finais

1. Qual é a principal finalidade de um pipeline de CI/CD?

Automatizar a integração, os testes e a entrega do software, garantindo mais rapidez e qualidade no desenvolvimento.

2. Qual é a diferença básica entre teste unitário e teste de integração?

O teste unitário verifica uma parte isolada do código, enquanto o teste de integração verifica se diferentes partes do sistema funcionam corretamente juntas.

3. Qual ferramenta do material é indicada para testes unitários?

Jest.

4. Qual ferramenta do material é indicada para testes de interface/integração e ponta a ponta?
Cypress.

5. Para que o Lighthouse foi utilizado nesta atividade?
Para avaliar o desempenho, acessibilidade, boas práticas e SEO da aplicação.

6. O que significa um gargalo de performance?

uma parte do sistema que limita o desempenho, fazendo com que a aplicação fique mais lenta ou demore mais para responder.

21. Checklist antes de entregar

☐ Executei os testes simples da função calcularTotal.
☐ Observei o comportamento de um teste que falha.
☐ Relacionei a verificação automática ao conceito de CI/CD.
☐ Executei ou acompanhei uma auditoria no Lighthouse.
☐ Registrei a pontuação de Performance.
☐ Identifiquei pelo menos um gargalo ou melhoria.
☐ Analisei o cenário de teste de carga.
☐ Inseri os prints solicitados.
☐ Respondi às perguntas finais.

22. Critérios de avaliação

Critério	Valor
Compreensão de CI/CD	1,5
Testes simples e interpretação	2,0
Compreensão de testes unitários/integração	1,5
Uso e registro do Lighthouse	2,0
Análise de gargalos e melhoria	1,5
Interpretação de carga/performance	1,0
Organização, evidências e respostas	0,5
TOTAL	10,0

PARTE 3 - ORIENTAÇÕES AO PROFESSOR, GABARITO E PLANO B

1. Adaptação realizada

Os materiais originais abordam configuração de pipeline CI/CD, Jest, Cypress e testes de performance com Lighthouse, JMeter ou k6. Para manter a sequência viável em três aulas e reduzir dependências de instalação, a prática principal utiliza JavaScript simples no navegador e Lighthouse. Jest, Cypress, JMeter e k6 permanecem no conteúdo conceitual, com exemplos e comparação de uso.

2. O que foi validado

• A função `calcularTotal()` foi executada com Node.js e também em um navegador Chromium; o resultado para [100, 50] foi 150.

• Os dois `console.assert` corretos foram executados no navegador sem mensagens de falha.

• O teste intencional com resultado esperado 999 foi executado e produziu a mensagem de falha, como previsto na atividade.

• O arquivo `script.js` passou em `node --check`, confirmando a sintaxe JavaScript utilizada na prática.

• O workflow YAML foi analisado por parser YAML, teve sua estrutura conferida e a etapa `node --check script.js` foi executada localmente com sucesso.

• O exemplo de Jest teve sintaxe e lógica verificadas, mas não foi executado dentro do Jest porque essa ferramenta não é necessária na prática e sua instalação depende do ambiente externo.

• Cypress, Lighthouse, JMeter e k6 aparecem como ferramentas/conceitos; não há scripts obrigatórios dessas ferramentas no documento. A auditoria Lighthouse depende da página escolhida e do navegador do laboratório.

3. Gabarito das perguntas finais

- 1. Automatizar verificações, testes e etapas de entrega, aumentando previsibilidade e qualidade.
- 2. Unitário verifica uma parte isolada; integração verifica partes funcionando juntas.
- 3. Jest.
- 4. Cypress.
- 5. Para auditar a performance da página e identificar métricas e oportunidades de melhoria.
- 6. Um ponto que limita o desempenho e prejudica o restante da experiência.

4. Gabarito da simulação de carga

O sinal mais claro de degradação aparece em 500 usuários simultâneos, quando o tempo médio sobe para 4,9 s e a taxa de erros chega a 8%. A conclusão esperada é que o sistema precisa de investigação e otimização antes de suportar essa carga com boa experiência.

5. Dificuldades comuns

Confundir CI/CD com GitHub
GitHub pode hospedar o repositório e executar pipelines, mas CI/CD é o processo/prática de automação, não o site em si.

Achar que teste automatizado elimina todo teste manual
Explique que automação complementa outras formas de validação; ela é especialmente útil para verificações repetitivas.

Confundir unitário com E2E
Retome: unitário = uma peça; integração/E2E = peças/jornada trabalhando juntas.

Tratar Lighthouse como nota absoluta
O relatório é um diagnóstico. O importante é interpretar métricas e recomendações no contexto da página.

Achar que performance é só internet rápida

Performance também depende do peso e da organização dos recursos da aplicação.

Focar em culpados quando um teste falha
Direcione a turma para causa, impacto e correção, reforçando colaboração.

6. Perguntas de intervenção durante a prática

- O teste está verificando um resultado específico?
- O que mudaria se essa função começasse a retornar um valor incorreto?
- Qual etapa do pipeline deveria impedir que uma falha conhecida avance?
- Qual recomendação do Lighthouse parece ter maior impacto?
- O problema observado está no front-end, na rede ou ainda não temos evidência suficiente?
- O aumento de usuários alterou tempo de resposta ou taxa de erros?

7. Plano B

Se o laboratório estiver sem acesso ao Lighthouse ou a uma página publicada:

- Utilize prints de um relatório Lighthouse previamente preparado pelo professor.
- Peça aos alunos que identifiquem métricas e recomendações a partir desse relatório.
- Mantenha a prática de testes simples no Console, que funciona em uma página local.
- Use a tabela simulada de carga para trabalhar interpretação de performance sem JMeter/k6.
- Se o GitHub estiver indisponível, analise o YAML impresso/projetado apenas como fluxo conceitual.

8. Síntese final

Síntese
CI/CD automatiza verificações e entregas; testes automatizados ajudam a detectar regressões; testes unitários verificam partes isoladas, enquanto integração/E2E verificam o funcionamento conjunto; Lighthouse ajuda a diagnosticar performance; testes de carga observam o comportamento do sistema quando a quantidade de acessos aumenta. O objetivo da Semana 18 é conectar qualidade, automação e experiência do usuário.